

Algorithms

Lecture #43

Syed Hamed Raza

Complexity Theory

Complexity Theory

Complexity Theory

- So far in the course, we have been building up a “bag of tricks” for solving algorithmic problems.
- Hopefully you have a better idea of how to go about solving such problems.
- What sort of design paradigm should be used: divide-and-conquer, greedy, dynamic programming.

Complexity Theory

Complexity Theory

- What sort of data structures might be relevant: trees, heaps, graphs.
- What is the running time of the algorithm.
- All of this is fine if it helps you discover an acceptably efficient algorithm to solve your problem.

Complexity Theory

Complexity Theory

- The question that often arises in practice is that you have tried every trick in the book and nothing seems to work.
- Although your algorithm can solve small problems reasonably efficiently (e.g., $n \leq 20$), for the really large problems you want to solve, your algorithm never terminates.

Complexity Theory

Complexity Theory

- Although your algorithm can solve small problems reasonably efficiently (e.g., $n \leq 20$), for the really large problems you want to solve, your algorithm never terminates.
- When you analyze its running time, you realize that it is running in exponential time, perhaps $n^{\sqrt{n}}$, or 2^n , or 2^{2n} , or $n!$ or worse!.

Complexity Theory

Complexity Theory

- By the end of 60's, there was great success in finding efficient solutions to many combinatorial problems.
- But there was also a growing list of problems for which there seemed to be no known efficient algorithmic solutions.

Complexity Theory

Complexity Theory

- People began to wonder whether there was some unknown paradigm that would lead to a solution to there problems.
- Or perhaps some proof that these problems are inherently hard to solve and no algorithmic solutions exist that run under exponential time.

Complexity Theory

Complexity Theory

- Near the end of the 1960's, a remarkable discovery was made.
- Many of these hard problems were interrelated in the sense that if you could solve any one of them in polynomial time, then you could solve all of them in polynomial time.
- This discovery gave rise to the notion of NP-completeness.

Complexity Theory

Complexity Theory

- This area is a radical departure from what we have been doing because the emphasis will change.
- The goal is no longer to prove that a problem **can** be solved efficiently by presenting an algorithm for it.
- Instead we will be trying to show that a problem **cannot** be solved efficiently.

Complexity Theory

Complexity Theory

- Up until now all algorithms we have seen had the property that their worst-case running time are bounded above by some **polynomial** in n .
- A **polynomial time algorithm** is any algorithm that runs in $O(n^k)$ time.
- A problem is solvable in polynomial time if there is a polynomial time algorithm for it.

Complexity Theory

Complexity Theory

- Some functions that do not look like polynomials (such as $O(n \log n)$) are bounded above by polynomials (such as $O(n^2)$).
- Some functions that do look like polynomials are not.
- For example, suppose you have an algorithm that takes as input a graph of size n and an integer k and run in $O(n^k)$ time.

Complexity Theory

Complexity Theory

- Is this a polynomial time algorithm?
- No, because k is an input to the problem so the user is allowed to choose $k = n$, implying that the running time would be $O(n^n)$.
- $O(n^n)$ is surely not a polynomial in n .
- The important aspect is that the exponent must be a constant independent of n .

Decision Problems

Decision Problems

- Most of the problems we have discussed involve optimization of one form of another.
- Find the shortest path, find the minimum cost spanning tree, maximize the knapsack value.
- For rather technical reasons, the NP-complete problems we will discuss will be phrased as decision problems.

Decision Problems

Decision Problems

- A problem is called a *decision problem* if its output is a simple “yes” or “no” (or you may think of this as true/false, 0/1, accept/reject.)
- We will phrase many optimization problems as decision problems.
- For example, the MST decision problem would be:

Decision Problems

Decision Problems

- We will phrase many optimization problems as decision problems.
- For example, the MST decision problem would be:
- Given a weighted graph G and an integer k , does G have a spanning tree whose weight is at most k ?

Complexity Classes

Complexity Classes

Class P

- This is the set of all decision problems that can be *solved* in polynomial time.
- We will generally refer to these problems as being “easy” or “efficiently solvable”.

Complexity Classes

Complexity Classes

Class NP

- This is the set of all decision problems that can be **verified** in polynomial time.
- This class contains P as a subset.
- It also contains a number of problems that are believed to be very **"hard"** to solve.

Complexity Classes

Complexity Classes

Class NP

- The term “NP” does not mean “not polynomial”.
- Originally, the term meant “**non-deterministic polynomial**” but it is a bit more intuitive to explain the concept from the perspective of verification.

Complexity Classes

Complexity Classes

Class NP-hard

- In spite of its name, to say that a problem is NP-hard does not mean that it is hard to solve.
- Rather, it means that if we could solve this problem in polynomial time, then we could solve *all* NP problems in polynomial time.
- Note that for a problem to NP-hard, it does not have to be in the class NP.

Complexity Classes

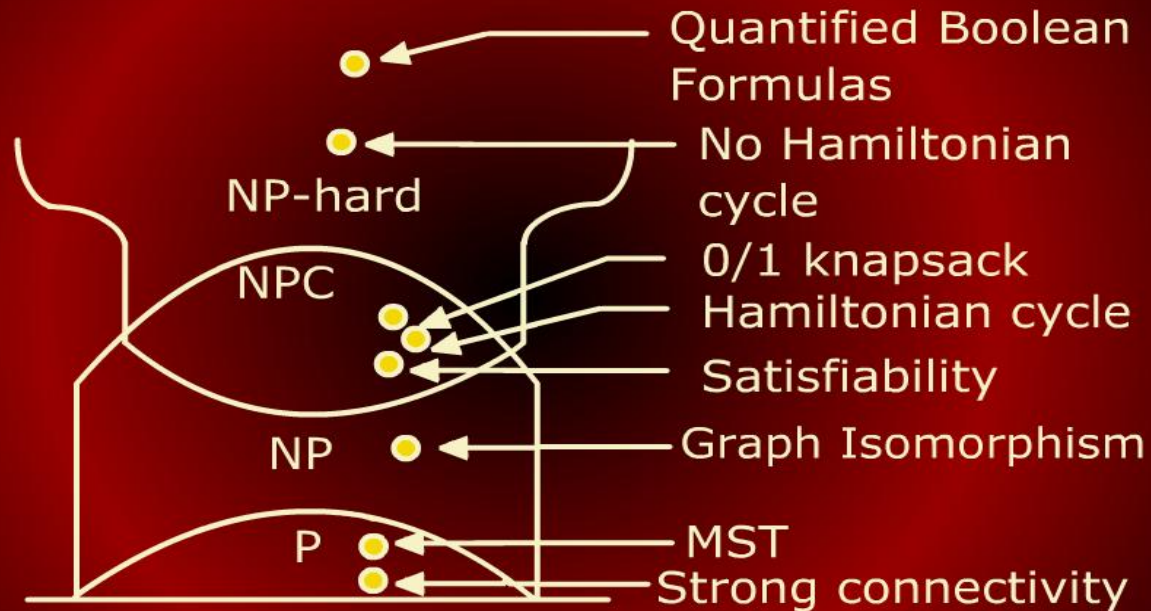
Complexity Classes

Class NP-complete

- A problem is NP-complete if
 1. it is in NP
 2. it is NP-hard

Complexity Classes

Complexity Classes



Polynomial Time Verification

Polynomial Time Verification

- Before talking about the class of NP-complete problems, it is important to introduce the notion of a **verification algorithm**.
- Many problems are hard to solve but they have the property that it is easy to verify the solution if one is provided.
- Consider the Hamiltonian cycle problem.

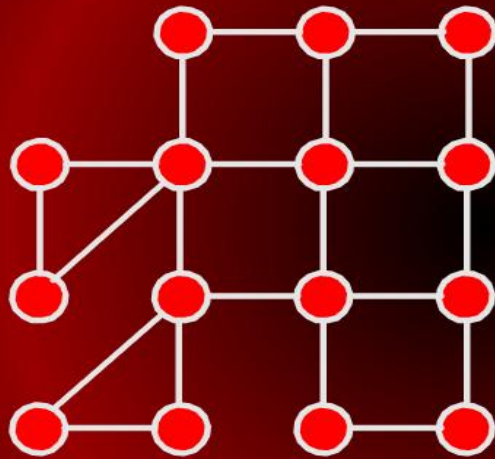
Polynomial Time Verification

Polynomial Time Verification

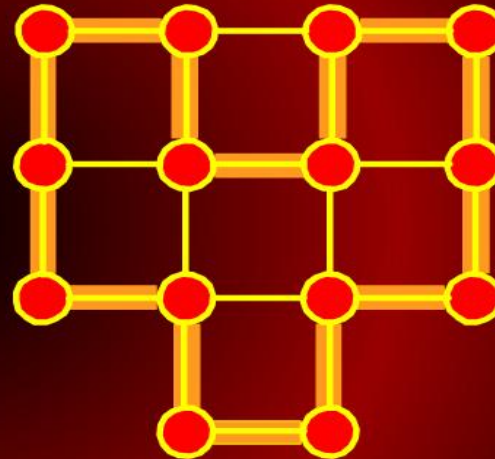
- Given an undirected graph G , does G have a cycle that visits every vertex exactly once?
- There is no known polynomial time algorithm for this problem.

Polynomial Time Verification

Polynomial Time Verification



Non-Hamiltonian



Hamiltonian

Polynomial Time Verification

Polynomial Time Verification

- However, suppose that a graph did have a Hamiltonian cycle.
- It would be easy for someone to convince of this.
- They would simply say: “the cycle is $\langle v_3, v_7, v_1, \dots, v_{13} \rangle$ ”

Polynomial Time Verification

Polynomial Time Verification

- We could then inspect the graph and check that this is indeed a legal cycle and that it visits all of the vertices of the graph exactly once.
- Thus, even though we know of no efficient way to *solve* the Hamiltonian cycle problem, there is a very efficient way to *verify* that a given cycle is indeed a Hamiltonian cycle.

Polynomial Time Verification

Polynomial Time Verification

- The piece of information that allows verification is called a *certificate*.
- Note that not all problems have the property that they are easy to verify.
- For example, consider the following two:
 1. $\text{UHC} = \{(G) \mid G \text{ has a unique Hamiltonian cycle}\}$
 2. $\overline{\text{HC}} = \{(G) \mid G \text{ has no Hamiltonian cycle}\}$

Polynomial Time Verification

Polynomial Time Verification

- Suppose that a graph G is in UHC.
- What information would someone give us that would allow us to verify this?
- They could give us an example of the unique Hamiltonian cycle and we could verify that it is a Hamiltonian cycle.
- But what sort of certificate could they give us to convince us that this is the *only* one?

Polynomial Time Verification

Polynomial Time Verification

- They could give another cycle that is not Hamiltonian.
- But this does not mean that there is not another cycle somewhere that is Hamiltonian.
- They could try to list every other cycle of length n , but this is not efficient at all since there are $n!$ possible cycles in general.

The Class NP

The Class NP

NP: set of all problems that can be verified by a polynomial time algorithm.

- Why is the set called “NP” and not “VP”?
- The original term NP stood for **non-deterministic polynomial time**.
- This referred to a program running on a non-deterministic computer that can make guesses.

The Class NP

The Class NP

- Such a computer could non-deterministically guess the value of the certificate.
- Then verify it in polynomial time.
- We have avoided introducing non-determinism here; it is covered in other courses such as automata or complexity theory.

The Class NP

The Class NP

- Observe that $P \subseteq NP$.
- In other words, if we can solve a problem in polynomial time, we can certainly verify the solution in polynomial time.
- More formally, we do not need to see a certificate to solve the problem; we can solve it in polynomial time anyway.

P=NP?

P=NP?

- However, it is not known whether $P = NP$.
- It seems unreasonable to think that this should be so.
- Being able to verify that you have a correct solution does not help you in finding the actual solution.
- The belief is that $P \neq NP$ but no one has a proof for this.

Reductions

Reductions

- The class NP-complete problems consists of a set of decision problems (a subset of class NP) that no one knows how to solve efficiently.
- But if there were a polynomial solution for even a single NP-complete problem, then every problem in NPC will be solvable in polynomial time.
- For this, we need the concept of **reductions**.